

Database Corruption Fix

Error

SQLITE_CORRUPT: database disk image is malformed

This happens when saving posts with certain markdown content.

Quick Fix (Most Common)

Option 1: Reset Database (Fresh Start)

```
cd backend
```

```
# Stop the backend server (Ctrl+C)
```

```
# Delete corrupted database
```

```
rm -f local.db
```

```
# Restart backend (will recreate database)
```

```
cargo run --bin setup # Recreate admin user
```

```
cargo run # Start server
```

Result: Fresh database, but **loses all data**.

Option 2: Use Turso Cloud (Recommended)

The corruption likely happens with local SQLite. Turso cloud is more robust.

1. Create Turso database:

```
# Install Turso CLI
```

```
curl -sSfL https://get.tur.so/install.sh | bash
```

```
# Login
```

```
turso auth login
```

```
# Create database
```

```
turso db create blog-db
```

```
# Get credentials
```

```
turso db show blog-db --url
```

```
turso db tokens create blog-db
```

2. Update backend/.env:

```
TURSO_DATABASE_URL=libsql://your-db.turso.io
TURSO_AUTH_TOKEN=your-token-here
```

3. Restart backend:

```
cd backend
cargo run --bin setup # Initialize database
cargo run              # Start server
```

Result: More reliable, cloud-backed storage.

Root Cause Analysis

Likely Causes:

1. Special Characters in Markdown

Markdown with certain characters might break SQL:

```
# Post with ' apostrophes
Content with "quotes" and `backticks`
Code blocks with special chars: \n \r \t
```

2. Large Content

Very long markdown posts might exceed limits.

3. Concurrent Writes

Multiple updates at once can corrupt local SQLite.

4. Filesystem Issues

Disk errors or improper shutdowns.

Prevention: Prepared Statements

Check if backend uses prepared statements:

The backend SHOULD be using libsql's parameter binding, which prevents this.

Verify in backend/src/repository.rs:

```
// GOOD (safe):
conn.execute(
    "UPDATE posts SET content = ? WHERE id = ?",
    libsql::params![content, id]
).await

// BAD (unsafe):
```

```
conn.execute(
    &format!("UPDATE posts SET content = '{}'" WHERE id = '{}'", content, id)
).await
```

Testing with Problematic Markdown

Try saving this markdown to test:

```
# Test Post
```

Here's some content with:

- Single quotes: It's working
- Double quotes: "Hello world"
- Backticks: `code here`
- Backslashes: C:\path\to\file
- Special chars: @#%^&*()

```
## Code Block
```

```
```javascript
function test() {
 const str = "test's \"quoted\" string";
 return `template ${str}`;
}
```

## More Special Cases

Math:  $2 + 2 = 4$ , 50% off Symbols: © ® ™ € £ ¥ Emoji:

Done!

```
If this saves successfully: Database and escaping work correctly.
If it fails: Need to fix parameter binding.
```

```

```

```
Manual Database Recovery
```

If you need to recover data from corrupted database:

```
1. Try SQLite Recovery
```

```
```bash
cd backend
```

```
# Attempt to dump data
sqlite3 local.db ".dump" > backup.sql

# Create new database from dump
sqlite3 local_new.db < backup.sql

# Replace old database
mv local.db local_corrupt.db
mv local_new.db local.db
```

2. Export Posts to JSON (if backend still runs)

```
# While backend is running
curl http://localhost:3001/api/posts?limit=1000 > posts_backup.json
```

Then recreate database and re-import.

Turso Cloud Migration (Detailed)

Step 1: Create Turso Database

```
# Install Turso CLI
curl -sSfL https://get.turso.so/install.sh | bash

# Login to Turso
turso auth login

# Create your database
turso db create blog-production

# Get connection details
turso db show blog-production --url
# Output: libsql://blog-production-username.turso.io

turso db tokens create blog-production
# Output: eyJhbGciOiJIJFZERTQSIzInR5cCI6IkpXVCJ9...
```

Step 2: Update Configuration

Edit backend/.env:

```
# Replace these values
TURSO_DATABASE_URL=libsql://blog-production-username.turso.io
TURSO_AUTH_TOKEN=eyJhbGciOiJIJFZERTQSIzInR5cCI6IkpXVCJ9...

# Keep these the same
```

```
PORT=3001
JWT_SECRET=your-secret-key
```

Step 3: Initialize Database

```
cd backend

# Clean rebuild
cargo clean
cargo build

# Initialize schema
cargo run --bin setup

# When prompted:
Email: admin@example.com
Password: your-secure-password
```

Step 4: Verify Connection

```
# Start backend
cargo run

# Test API (new terminal)
curl http://localhost:3001/health
# Should return: {"status":"ok"}

# Verify database
turso db shell blog-production
# SQL prompt opens

SELECT COUNT(*) FROM posts;
# Should show 0 or more posts

.exit
```

Frontend Changes (None Needed)

The frontend doesn't need any changes. It connects to the same backend API regardless of where the database is stored.

Monitoring Database Health

Check Turso Database Status

```
# List databases
turso db list

# Show database info
turso db show blog-production

# Check database size
turso db inspect blog-production
```

Database Limits (Turso Free Tier)

- **Storage:** 9 GB
- **Rows Read:** 1 billion/month
- **Rows Written:** 25 million/month
- **Databases:** 500

More than enough for a blog!

Quick Markdown Sanitization (Temporary Fix)

If you want to keep using local SQLite, add this to frontend before saving:

app/admin/posts/[id]/edit/page.tsx:

```
const sanitizeMarkdown = (content: string) => {
  // Remove potential problem characters
  return content
    .replace(/\u0000/g, '') // Remove null bytes
    .trim()
}

const handleSubmit = async (e: React.FormEvent) => {
  // ...
  const postData = {
    // ...
    content: sanitizeMarkdown(formData.content),
  }
}
```

Note: This is a workaround. Proper fix is using Turso or fixing the backend.

Debugging Tips

Enable SQLite Debugging

```
cd backend

# Run with debug logging
RUST_LOG=debug,libsql=trace cargo run

# Try saving a post
# Watch for SQL errors in console
```

Check Database File

```
cd backend

# Check if database file exists
ls -lh local.db

# Check file integrity
sqlite3 local.db "PRAGMA integrity_check;"
# Should return: ok

# If corrupted:
sqlite3 local.db "PRAGMA integrity_check;"
# Returns: *** in database main ***
```

Complete Fresh Start (Nuclear Option)

If nothing else works:

```
cd blog-system

# Backend: Remove all database files
cd backend
rm -f local.db local.db-shm local.db-wal
cargo clean

# Frontend: Clear cache
cd ../frontend
rm -rf .next node_modules/.cache

# Rebuild everything
cd ../backend
cargo build
cargo run --bin setup
```

```
# New terminal
cd ../frontend
npm run dev
```

Recommended Solution

For Production/Reliability: → Use **Turso Cloud** (free tier is generous)

For Development/Testing: → Use Turso or reset local DB frequently

Why Turso? - No corruption issues - Cloud-backed (no local file) - Free tier (9GB, 1B reads/month) - Global replication - Automatic backups - Better concurrency handling

Summary

Issue	Quick Fix	Long-term Fix
Database corrupted	Delete <code>local.db</code> , restart	Use Turso Cloud
Markdown breaks SQL	Sanitize input	Use Turso Cloud
Concurrent writes	Restart server	Use Turso Cloud
Large posts fail	Chunk content	Use Turso Cloud

Best Solution: Migrate to Turso Cloud (10 minutes setup, prevents all these issues)

Turso Setup Script

Save this as `setup-turso.sh`:

```
#!/bin/bash

echo " Setting up Turso for your blog"

# Check if turso is installed
if ! command -v turso &> /dev/null; then
    echo " Installing Turso CLI..."
    curl -sSfL https://get.tur.so/install.sh | bash
fi

# Login
```



```

echo " Login to Turso..."
turso auth login

# Create database
echo " Creating database..."
turso db create blog-production

# Get credentials
echo " Getting credentials..."
URL=$(turso db show blog-production --url)
TOKEN=$(turso db tokens create blog-production)

# Update .env
echo " Updating backend/.env..."
cat > backend/.env << EOF
TURSO_DATABASE_URL=$URL
TURSO_AUTH_TOKEN=$TOKEN
PORT=3001
JWT_SECRET=$(openssl rand -base64 32)
EOF

echo " Turso setup complete!"
echo ""
echo "Next steps:"
echo "1. cd backend"
echo "2. cargo run --bin setup"
echo "3. cargo run"

```

Run it:

```

chmod +x setup-turso.sh
./setup-turso.sh

```

Choose your path: - **Quick:** Delete local.db and restart - **Reliable:** Set up Turso Cloud (recommended) - **Debug:** Check backend SQL queries

All paths lead to a working blog!